Programming Concepts and Paradigms (PCP)

Programmierübung zu Clojure Woche 3

Hauptthemen: Lazyness, Scopes, for-Comprehensions, Records
Zeitfenster: ca. 4-5 Lektionen

1. Aufgabe

Im Input wurde die folgende Funktion gezeigt:

```
(defn sum-numbers [nums]
  (map println nums) ; println wird nie ausgeführt
  (reduce + nums))
```

Warum wird **println** nie ausgeführt?

Finden Sie zwei Möglichkeiten in *Clojure*, wie Sie die Ausgabe der Zahlen mit **println** dennoch erreichen können.

2. Aufgabe

Was ergibt der folgende Ausdruck? Erklären Sie nachvollziehbar, wie das Ergebnis zustande kommt.

```
(let [a 1 b 2 c 3]
  (let [b c]
    (let [a b]
      (let [c a]
        [a b c]))))
```

3. Aufgabe

Nutzen Sie `iterate` oder andere lazy-Funktionen, um die folgenden unendlichen Zahlenfolgen zu generieren:

- 2, 4, 6, 8, 10, 12, …
- 3, 9, 27, 81, 243, …
- 1, 3, 7, 15, 31, 63, 127, …
- 1, 1, 2, 3, 5, 8, 13, 21, 34, …

Testen Sie dabei jeweils mit `take`, ob die ersten 20-30 Zahlen korrekt sind.

4. Aufgabe

Schreiben Sie die folgenden drei Ausdrücke als for-Comprehensions:

```
(mapcat (fn [x] (map #(+ x %) (range 3))) (range 3))

(map #(list % %) (filter #(== (mod % 3) 0) (range 10)))

(filter #(> (second %) (first %))
        (mapcat (fn [x] (map #(list x %) (range 4))) (range 4)))
```

5. Aufgabe

Finden Sie alle Zahlen a, b, c im Bereich [1 .. 100], welche die *pythagoräische Gleichung*

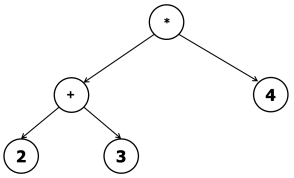
$$a^2 + b^2 = c^2$$

erfüllen.

Geben Sie dabei eine Lösung mit einer for-Comprehension an.

6. Aufgabe

Im Folgenden wollen wir Bäume nutzen, um einfache mathematische Ausdrücke zu speichern. Der Ausdruck (2 + 3) ∗ 4 kann z.B. wie folgt als Baum repräsentiert werden:



Im Github-Repository finden Sie eine Datei *aufgabe6.clj*. Dort wird gezeigt, wie solche Bäume in *Clojure* mit einigen Record-Definitionen repräsentiert werden können (beachten Sie, dass ebenfalls die Datei *parser.clj* benötigt wird).

Der hier abgebildete Baum kann z.B. wie folgt mit den gegebenen Datenstrukturen umgesetzt werden (**Op** repräsentiert dabei binäre Operatoren und **Val** Werte):

```
(defrecord Op [op left right])    ; bereits in parser.clj definiert
(defrecord Val [value])

; Baum für (2+3) * 4
(def expr (Op. * (Op. + (Val. 2) (Val. 3)) (Val. 4)))
```

a.) Zeichnen Sie den Baum für den Ausdruck 2 * (4-2) + (6/3).

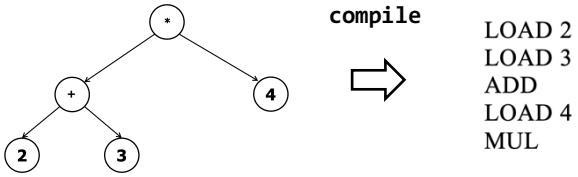
b.) Erzeugen Sie den Baum für Ausdruck aus a.) von Hand, indem Sie wie im Beispiel **Op**- und **Val**-Knoten erzeugen.

c.) Nutzen Sie die parse-Funktion, um den Baum aus a.) zu erzeugen und vergleichen Sie Ihren Baum mit dem Baum von der **parse**-Funktion (die Bäume können mit = verglichen werden und sollten identisch sein).

d.) Implementieren Sie eine Funktion **eval-expr**, welche den Wert eines Ausdrucks rekursiv berechnet. Dazu folgende Hinweise: In diese Aufgabe reicht es aus, mit ganzen Zahlen zu rechnen, und die Division kann ebenfalls ganzzahlig rechnen.

e.) Implementieren Sie eine Funktion **compile-expr**, welche den Baum in ein Programm für die Stack-Maschine aus der Woche 2 übersetzt.

Testen Sie Ihre Funktion mit mehreren Ausdrücken und stellen Sie sicher, dass die **eval-expr**-Funktion aus d.) und die Stack-Maschinen-Programme zum jeweiligen Ausdruck dieselben Ergebnisse liefern, indem Sie die Programme auf der Stack-Maschine ausführen.



7. Aufgabe

Gegeben ist ein globales Verzeichnis mit Vornamen und Telefonnummer von Personen:

```
(def my-phone-dir
  (atom
   [{:name "Anna" :phone 1234}
    {:name "Ben" :phone 5678}]))
```

Ein Programm soll zwei Möglichkeiten bieten:

1. Suchen der Nummer anhand des Vornamens
2. Hinzufügen einer neuen Person mittels Vornamen und Nummer im Verzeichnis

Eine mögliche Interaktion könnte dann wie folgt verlaufen:

```
(look-at my-phone-dir "Anna")        ; 1234
(look-at my-phone-dir "Ben")         ; 5678

(look-at my-phone-dir "Alex")        ; nil
(add-entry my-phone-dir "Alex" 2468) ; nil
(look-at my-phone-dir "Alex")        ; 2468
```

a) Warum widerspricht diese Interaktion fundamental unserem bisherigen funktionalen Programmiergrundsatz?

b) Schreiben Sie die Funktion `look-at` für das Suchen der Nummer anhand des Vornamens

c) Schreiben Sie die Funktion `add-entry` für das Hinzufügen einer neuen Person mit Vornamen und Nummer.